

Hosting di Oracle Cloud (Gratis, ARM) — Awal sampai Akhir

Hosting on Oracle Cloud (Free, ARM) — Start to Finish

Tugas Besar Big Data · ETL & ELT + Dashboard

ID: Panduan men-*hosting* seluruh stack (Kafka + Spark + Hive) di VM **Oracle Cloud** "Always Free" (Ampere ARM, 24 GB RAM) supaya bisa diakses dari internet — **gratis**.
EN: Guide to host the whole stack (Kafka + Spark + Hive) on an **Oracle Cloud** "Always Free" (Ampere ARM, 24 GB RAM) VM so it is reachable from the internet — **free**.

■ Model Keamanan / Security Model (BACA DULU / READ FIRST)

ID: Layanan stack ini (HiveServer2, Spark, Kafka) **tanpa autentikasi & plaintext** — berbahaya jika dibuka langsung ke internet (siapa pun bisa DROP tabel, jalankan job Spark sembarang = eksekusi kode jarak jauh, baca/tulis Kafka). **Solusi (default repo ini): jangan buka port layanan ke internet sama sekali.** Sebagai gantinya:

1. Hanya port SSH (22) yang terbuka ke publik — dibatasi ke IP kamu.
2. Semua port layanan di-bind ke `127.0.0.1` di dalam VM (`docker-compose.yml`), jadi tidak ada di antarmuka publik.
3. Akses dari luar lewat SSH tunnel — terenkripsi + terautentikasi dengan kunci SSH (lihat §8).
4. VM di-hardening (kunci SSH saja, fail2ban, auto-update — \$A) dan `.env` mode `600`.

Lapisan ini disebut *defense in depth*: meski satu lapis bocor, lapis lain masih melindungi. Membuka port langsung ke `0.0.0.0` hanya lewat override **insecure** yang eksplisit (`docker-compose.prod.yml`) — hindari kecuali untuk demo sekali pakai.

EN: the stack services are **no-auth / plaintext** — dangerous if exposed directly. **This repo's default keeps them OFF the public internet:** only **SSH (22)** is open (locked to your IP), all service ports **bind to 127.0.0.1** inside the VM, and you reach them through an **SSH tunnel** (encrypted + key-authenticated, §8). The VM is hardened (\$A) and `.env` is `chmod 600`. Direct public exposure is only via the explicit **insecure** `docker-compose.prod.yml` override — avoid it except for a throwaway demo.

0. Kenapa Oracle? / Why Oracle?

ID: Stack butuh ~7.5 GB RAM (caveat #6). Free tier AWS/GCP/Azure cuma ~1 GB → tidak cukup.
Oracle "Always Free" Ampere A1 memberi **4 OCPU + 24 GB RAM gratis selamanya** — satu-satunya opsi gratis yang muat. Catatannya: arsitektur **ARM/aarch64**.
EN: The stack needs ~7.5 GB RAM; AWS/GCP/Azure free micro tiers (~1 GB) are too small. Oracle's Always-Free Ampere A1 gives 4 OCPU + 24 GB RAM free forever — the only free option that fits. Catch: it's **ARM/aarch64**.

Kompatibilitas ARM (sudah diverifikasi) / ARM compatibility (verified): image `apache/hive:4.0.0`, `apache/kafka:3.9.0`, `postgres:16`, `provectuslabs/kafka-ui` **sudah arm64**. Hanya image Spark yang diganti: `bitnami/legacy/spark` (tanpa arm64) → **`apache/spark:3.5.3-python3`** (resmi, multi-arch). Perubahan ini **sudah diterapkan** di `docker/Dockerfile.spark` & `docker-compose.yml`.

1. Buat VM / Provision the VM (Oracle Console)

1. Daftar akun Oracle Cloud gratis. Pilih *home region* yang masih punya kapasitas ARM.
(Jika "Out of capacity", ulangi beberapa saat / coba AD/region lain — ini friksi yang umum.)
2. **Compute** → **Instances** → **Create instance**:
 - **Shape**: VM.Standard.A1.Flex → **4 OCPU, 24 GB RAM**.
 - **Image**: Canonical **Ubuntu 24.04 (aarch64)**.
 - **Boot volume**: ~**120 GB** (jatah always-free 200 GB; default 50 GB terlalu sempit untuk image Spark + deps + warehouse).
 - **SSH key**: unggah kunci publik kamu.
3. Catat **Public IP** instance.

2. Jaringan — Mode Aman (hanya SSH) / Network — Secure Mode (SSH only)

ID: Di mode aman **hanya port 22 (SSH)** yang dibuka ke internet. Semua layanan lain dicapai lewat SSH tunnel (§8), jadi tidak perlu membuka 10000/9092/8080/dst ke publik.

EN: In secure mode **only port 22 (SSH)** is open to the internet; every other service is reached through an SSH tunnel (§8), so you never open 10000/9092/8080/etc. publicly.

2a. Cloud firewall (VCN Security List / NSG)

Networking → VCN → Subnet → Security List → **Add Ingress Rules**:

- **TCP 22**, Source CIDR = **IP kamu / your IP** (mis. 203.0.113.5/32). *Hindari 0.0.0.0/0*.
- (Tidak ada rule lain — tidak perlu / no other rules needed.)

2b. Host firewall (ufw — default deny)

```
sudo ufw default deny incoming sudo ufw default allow outgoing sudo ufw limit 22/tcp # 'limit' = rate-limit
SSH (anti brute-force) sudo ufw --force enable sudo ufw status verbose # verifikasi: hanya 22 yang ACCEPT
```

ID: Jangan buka port layanan di sini — biarkan tertutup; SSH tunnel yang akan menjembatani.

EN: do NOT open the service ports here — keep them closed; the SSH tunnel bridges them.

(Opsi insecure / insecure option) Jika benar-benar perlu ekspos langsung untuk demo: tambah ingress + ufw allow untuk 7077, 8080, 8085, 9083, 9092, 10000, 10002 **dibatasi ke IP kamu**, lalu jalankan dengan override docker-compose.prod.yml (§6). Tetap berisiko — matikan setelah demo.

3. Install Docker di VM / Install Docker on the VM

```
sudo apt-get update sudo apt-get install -y ca-certificates curl sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc sudo chmod a+r
/etc/apt/keyrings/docker.asc echo "deb [arch=arm64 signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" \ | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null sudo apt-get update sudo apt-get install -y docker-ce
docker-ce-cli containerd.io docker-compose-plugin sudo usermod -aG docker $USER && newgrp docker docker
version && docker compose version
```

4. Kirim Proyek ke VM / Ship the Project to the VM

Opsi A — via GitHub (rekomendasi):

```
# di komputer lokal / on your local machine – JANGAN commit .env (kunci FRED): git init && git add . && git commit -m "Big Data project" git remote add origin https://github.com/<user>/<repo>.git && git push -u origin main # di VM / on the VM: git clone https://github.com/<user>/<repo>.git && cd <repo>
```

Opsi B — via scp:

```
scp -r "Big_Data_Tubes" ubuntu@<PUBLIC_IP>:~/bigdata
```

ID: Pastikan file input ikut: raw/synthetic_credit_clients.csv (sumber kredit 1) & raw/fred_*_2005.json. **EN:** make sure the raw inputs are present.

5. Konfigurasi .env di VM / Configure .env on the VM

```
cp .env.example .env nano .env # FRED_API_KEY=<kunci_fred_kamu> # PUBLIC_IP=<PUBLIC_IP_VM> # dipakai listener EXTERNAL Kafka
```

6. Build & Jalankan Stack / Build & Run the Stack

ID: Mode **aman** = file base saja (port ke 127.0.0.1). **Jangan** pakai `docker-compose.prod.yml` kecuali sengaja mau ekspos publik (insecure).

EN: **Secure** mode = base file only (ports on 127.0.0.1). Do **not** add `docker-compose.prod.yml` unless you deliberately want public exposure (insecure).

```
docker compose build # pertama kali agak lama (ARM) / first build is slow on ARM docker compose up -d docker compose ps # semua harus Up / all Up docker compose logs hive-metastore --tail 5 # cari "Initialized schema successfully"
```

ID: Sekali saja, buat volume warehouse bisa ditulis (caveat #5):

```
bash docker run --rm --user 0 --entrypoint sh -v bigdata-tubes_warehouse:/wh bigdata-spark:3.5 -c "chmod -R 777 /wh"
```

(Insecure, opsional) untuk ekspos langsung ke publik:

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d (set PUBLIC_IP di .env).
```

7. Isi Warehouse (jalankan pipeline) / Populate the Warehouse (run the pipeline)

ID: CTGAN **di-skip** (file sintetis sudah ada). Tahap yang menulis warehouse pakai `-u root`.

```
docker compose exec spark-master python /app/etl_pipeline/extract.py # FRED -> raw/ (butuh FRED_API_KEY)
docker compose exec spark-master spark-submit /app/etl_pipeline/transform.py docker compose exec -u root
spark-master spark-submit /app/etl_pipeline/load.py docker compose exec -u root spark-master spark-submit
/app/elt_pipeline/extract_load.py docker compose exec -u root spark-master spark-submit
/app/elt_pipeline/run_transform.py docker compose exec -u root spark-master spark-submit
/app/dashboard/build_dashboard_kpis.py
```

Smoke test (beeline):

```
docker compose exec hiveserver2 beeline -u "jdbc:hive2://localhost:10000" \ -e "SHOW TABLES IN bigdata_elt;
SELECT * FROM bigdata_elt.kpi_overall;" # Hasil terverifikasi: default_rate 0.2212, total_clients 150000
```

8. Akses Aman via SSH Tunnel / Secure Access via SSH Tunnel

ID: Karena port layanan hanya di 127.0.0.1 pada VM, kita "tarik" ke komputer lokal lewat SSH tunnel. Lalu akses semuanya seolah di localhost — terenkripsi & terautentikasi kunci SSH.
EN: service ports live on the VM's 127.0.0.1, so we forward them to your machine over SSH and use everything as if it were localhost — encrypted and SSH-key-authenticated.

Satu perintah, semua port / one command, all ports (jalankan di komputer lokal, biarkan terbuka):

```
ssh -N \ -L 10000:localhost:10000 \ # HiveServer2 JDBC/ODBC (Tableau / Power BI) -L 10002:localhost:10002 \ #  
HiveServer2 UI -L 8080:localhost:8080 \ # Spark master UI -L 8085:localhost:8085 \ # Kafka UI -L  
9092:localhost:9092 \ # Kafka broker ubuntu@<PUBLIC_IP>
```

Windows (PowerShell) sama persis — OpenSSH sudah bawaan Windows 10/11.

Selama tunnel terbuka / while the tunnel is open:

```
| Layanan / Service | URL lokal / local URL |  
|---|---|  
| Spark master UI | http://localhost:8080 |  
| Kafka UI | http://localhost:8085 |  
| HiveServer2 UI | http://localhost:10002 |  
| HiveServer2 JDBC/ODBC | localhost:10000 |
```

Dashboard (Tableau / Power BI) via Hive ODBC — lewat tunnel

DSN **tetap pakai localhost** (bukan IP publik), karena tunnel sudah meneruskannya:

- Host localhost, Port 10000, Database bigdata_etl (Tableau) / bigdata_elt (Power BI)
- Hive Server Type **HiveServer2**, Mechanism **User Name**, User hive
- **Thrift Transport = Binary, SSL = OFF** → Test → SUCCESS
- Power BI: Get Data → ODBC → [DSN] → centang kpi_* → Load.

ID: Keunggulan: tidak ada port layanan terbuka di internet, lalu lintas dienkripsi SSH, dan hanya pemegang kunci SSH yang bisa connect — tanpa mengubah konfigurasi Hive/Spark/Kafka.
EN: no service ports on the internet, SSH-encrypted traffic, only SSH-key holders can connect — with zero changes to Hive/Spark/Kafka configs.

A. Hardening VM / Harden the VM (lapisan tambahan / extra layers)

ID: Lakukan sekali setelah login pertama. **EN:** do this once after first login.

1. SSH kunci-saja (matikan password) / SSH key-only (disable passwords):

```
sudo sed -i 's/^#\?PasswordAuthentication.*\/PasswordAuthentication no/' /etc/ssh/sshd_config sudo sed -i  
's/^#\?PermitRootLogin.*\/PermitRootLogin no/' /etc/ssh/sshd_config sudo systemctl restart ssh
```

2. fail2ban (blok brute-force SSH) / block SSH brute-force:

```
sudo apt-get install -y fail2ban sudo systemctl enable --now fail2ban
```

3. Update keamanan otomatis / automatic security updates:

```
sudo apt-get install -y unattended-upgrades sudo dpkg-reconfigure -f noninteractive unattended-upgrades
```

4. Izin file rahasia / secrets file permissions:

```
chmod 600 .env # kunci FRED hanya bisa dibaca pemilik / owner-only
```

5. (Optional, ekspos browser dengan TLS) / (Optional, expose UIs with TLS): jika butuh UI di browser tanpa tunnel, taruh **reverse proxy Caddy** (HTTPS otomatis + Basic Auth) di depan port web (8080/8085/10002) dan buka **hanya 443** ke IP kamu. Caddyfile minimal:

```
spark.contoh.com { # arahkan domain -> PUBLIC_IP basicauth { admin <hash bcrypt> } reverse_proxy localhost:8080 }
```

ID: Untuk HiveServer2/ODBC tetap pakai SSH tunnel (protokol Thrift biner, bukan HTTP).
EN: keep HiveServer2/ODBC on the SSH tunnel (Thrift binary, not HTTP).

6. (Optional lanjut) Autentikasi HiveServer2: set `hive.server2.authentication=LDAP` (butuh server LDAP) atau `CUSTOM` (kelas Java) untuk user/password di level Hive. Untuk tugas ini, SSH tunnel sudah cukup; ini bonus *defense in depth*.

9. Jika Build Spark ARM Bermasalah / If the Spark ARM Build Fails (fallback QEMU)

ID: Jika rebase `apache/spark` bikin masalah, jalankan image amd64 lama via emulasi (lebih lambat, tapi sekali jalan). **EN:** if the `apache/spark` rebase misbehaves, run the old amd64 image under emulation (slower, but it's a one-time load).

```
docker run --privileged --rm tonistiigi/binfmt --install all # aktifkan emulasi amd64 di ARM
```

Lalu di `docker/Dockerfile.spark` kembalikan FROM `bitnamilegacy/spark:3.5` dan tambahkan `platform: linux/amd64` pada service `spark-master/spark-worker`. Hive/Kafka/Postgres tetap arm64 native.

10. Matikan / Teardown (PENTING untuk keamanan / IMPORTANT for security)

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml down -v # stop + hapus volume
```

Lalu di Oracle Console: **Instance** → **Terminate** (centang hapus boot volume) agar tidak ada layanan tanpa-auth yang tetap terbuka di internet.

EN: then terminate the Oracle instance (delete the boot volume) so no no-auth service stays exposed.

Ringkasan Urutan / Order Summary (mode aman / secure mode)

1. Buat VM ARM (24 GB) → 2. Firewall: **hanya SSH/22** ke IP kamu (VCN + ufw default-deny) →
2. Install Docker → **SA hardening** (SSH kunci-saja, fail2ban, auto-update, `chmod 600 .env`) →
3. Kirim repo + isi `.env` (`FRED_API_KEY`; `PUBLIC_IP` hanya untuk mode insecure) →
4. `docker compose build + up -d` (**base saja**, port di `127.0.0.1`) → 6. `chmod warehouse` →
5. jalankan pipeline (`-u root`) → 8. **buka SSH tunnel** → konek dashboard ke `localhost:10000` →
6. **teardown** setelah demo (`down -v` + terminate instance).